```
// Methods for a 1 output node, 1 hidden layer backpropagation network
// without biases

#include "stdafx.h" // For MSVC, must be first!

#include "bareprop.h"

// Default constructor
BareProp::BareProp()
{
   objType = "BareProp";
   biasFlag = false; // BareProp networks don't have bias nodes
}

// Copy constructor
BareProp::BareProp( const BareProp& rhs )
{
   BareProp::copy( rhs ); // use the copy utility
}

// Overloaded = operator
BareProp& BareProp::operator= ( const BareProp& rhs )
{
   if ( &rhs != this ) // check for self-assignment
      BareProp::copy( rhs ); // use the copy utility

   return *this; // enables A = B = C
}

// Copy utility
void BareProp::copy( const BareProp& rhs )
{
   Network::copy( rhs ); // call immediate base object copy
   nHidden = rhs.nHidden;
   hW = rhs.hW;
   hWup = rhs.hWup;
   y = rhs.y;
   I = rhs.I;
   hO = rhs.hO;
   oW = rhs.oW;
   h_err = rhs.h_err;
   o_err = rhs.o_err;
   hG = rhs.hG;
   oG = rhs.oG;
}

// Loads a DataSet object into the BareProp Model
void BareProp::setDataSet( DataSet& dataObj )
{
   theData = dataObj; // set "theData" to incoming DataSet

   // Easier on the eyes
   unsigned nInput = theData.getInput();

   // Size the vectors for inputs
   I.resize( nInput ); // holds inputs from one exemplar

   // Use Model utility function to extract input Matrices
   Model::extractInputMatrices();

   // If training set is loaded, size BareProp training Matrices and vectors
   if ( theData.trainLoaded() )
   {
      // Get the output vector y from the training set's last column
      y = theData.getTrainMatrix().col( nInput );

      // Build the DataSet TwoSet object for the training set
      if ( theData.getDiscrete() ) // TwoSet object makes sense only for discrete output
```

```
         theData.setTrainTwoSet();
   }

   // If test set is loaded & discrete, build the TwoSet object for the test set
   if ( theData.testLoaded() && theData.getDiscrete() )
      theData.setTestTwoSet();
}

// Set the number of hidden nodes, note that training set must have been loaded!
void BareProp::setHidden( const unsigned n )
{
   // Easier on the eyes
   unsigned nInput = theData.getInput(); // number of input nodes

   // Hidden number must be nonzero, and training set must have been loaded
   assert ( n != 0 && theData.trainLoaded() );

   nHidden = n;

   // Size the hidden weight, update and gradient Matrices
   hW.resize( nHidden, nInput );
   hWup.resize( nHidden, nInput );
   hG.resize( nHidden, nInput );

   // Size the vectors for hidden outputs and weights
   hO.resize( nHidden ); // hidden node output vector
   oW.resize( nHidden ); // output weight vector
   oG.resize( nHidden ); // output weight gradient vector
   h_err.resize( nHidden ); // error terms for the hidden outputs

   builtFlag = true; // the object has now been formally constructed

   weightsSetFlag = false; // the weights have not yet been set
}

// Returns the degrees of freedom of this Network object
unsigned BareProp::df()
{
   return ( ( theData.getInput() + 1 ) * nHidden );
}

// Randomizes initial weights in this Network object
void BareProp::randomize()
{
   assert( builtFlag ); // the object must have been built first

   // Randomize the weights
   hW.random( randomLimit ); // randomize hW
   nvec::random( oW, randomLimit ); // randomize oW

   // Log to history file
   if ( historyFlag ) // make sure flag for history is set
   {
      // Construct the message to the output stream
      ostrstream fileStream;
      fileStream << "I've randomized the weights of a " << objType << " object."
         << endl << endl << ends;
      addHistory( fileStream ); // append to history file
   }

   weightsSetFlag = true; // set flag to indicate weights now set
}

// Save this Network object to a file, takes filename as ( string ) argument,
//    returns boolean flag to indicate if file succesfully saved
bool BareProp::save( string& filename )
{
   bool success = false; // flag to indicate file successfully saved
```

```
   // Open the output file to save data, overwrite file if it exists
   ofstream savefile( filename.c_str(), ios::out | ios::trunc );

   // Test to insure it was opened
   if ( !savefile.is_open() )
      cout << "Error in opening file to save " << objType << " network!" << endl;
   else
   {
      outputHeader( savefile ); // output the header to the file

      // Next line is hidden layer label, followed by hidden layer matrix
      savefile << "Hidden layer:" << endl;
      savefile << hW.setHeader( false ); // no header is necessary

      // Followed by output layer label, followed by output layer vector
      savefile << "Output layer:" << endl;
      savefile << oW << endl;

      // Print message to user notifying successful save to file
      cout << "The " << objType << " network was successfully saved to " << filename;
      cout << "." << endl;

      savefile.close(); // close output file

      success = true; // set flag to indicate file successfully saved

      // Log to history file
      if ( historyFlag ) // make sure flag for history is set
      {
         // Construct the output stream with object name and header
         ostrstream fileStream;
         fileStream << "I've saved the file " << filename << ":" << endl;
         outputHeader( fileStream ); // output the header to the history file
         fileStream << endl << ends;

         addHistory( fileStream ); // append to history file
      }
   }

   return success; // return flag to indicate if file successfully saved
}

// Load this Network object from a file, takes filename as ( string ) argument
//    returns boolean flag to indicate if file succesfully loaded
bool BareProp::load( string& filename )
{
   // Easier on the eyes
   unsigned nInput = theData.getInput(); // number of input nodes

   bool success = false; // flag to indicate file successfully loaded

   string lineString, // holder for strings pulled from file
      goodFile; // the name of the actual file

   // Open the file as read-only, and associate it with the 'label' loadfile
   goodFile = util::getGoodFile( filename );
   ifstream loadfile( goodFile.c_str(), ios::in );

   getline( loadfile, lineString ); // 1st line should be BareProp
   util::chopEndl( lineString ); // remove <cr> from end of string if exists
   assert( lineString == objType );

   getline( loadfile, lineString ); // "No bias nodes by definition"

   unsigned nInputFromFile; // number of input nodes obtained from file
   loadfile >> nInputFromFile; // retrieve number of input nodes
   getline( loadfile, lineString ); // " input nodes"
```

```
   // Make sure number of input nodes matches dataset
   if ( nInputFromFile != nInput )
   {
      cout << "I cannot load this file:" << endl;
      cout << "The number of input nodes do not match the dataset ( ";
      cout << nInput << " )" << endl;
   }
   else
   {
      getline( loadfile, lineString ); // "1 hidden layer by definition"

      loadfile >> nHidden; // retrieve number of hidden nodes
      getline( loadfile, lineString ); // " hidden nodes"
      setHidden( nHidden ); // set the architecture of this object

      getline( loadfile, lineString ); // "1 output node by definition"

      getline( loadfile, lineString ); // "Hidden layer:"
      loadfile >> hW; // retrieve hidden layer

      // Pick up the space and carriage return after the Matrix object
      getline( loadfile, lineString ); // THIS IS CRITICAL!

      getline( loadfile, lineString ); // "Output layer:"
      loadfile >> oW; // retrieve output layer

      weightsSetFlag = true; // set flag to indicate weights now set

      outputHeader( cout ); // report to user
      cout << "I've loaded the file." << endl;

      // Log to history file
      if ( historyFlag ) // make sure flag for history is set
      {
         // Construct the output stream with object name and header
         ostrstream fileStream;
         fileStream << "I've loaded the file " << goodFile << ":" << endl;
         outputHeader( fileStream ); // output header to history file
         fileStream << endl << ends;

         addHistory( fileStream ); // append to history file
      }

      success = true; // set flag to indicate file successfully loaded
   }

   loadfile.close(); // close input file

   return success; // return flag to indicate if file successfully loaded
}

// Outputs a header to ostream describing this BareProp model architecture
void BareProp::outputHeader( ostream& outputStream )
{
   // Easier on the eyes
   unsigned nInput = theData.getInput(); // number of input nodes

   outputStream << objType << endl; // type of object

   // Remind user biases by definition
   outputStream << "No bias nodes by definition" << endl;

   // Next line is number of input nodes inherited from model object
   outputStream << nInput << " input nodes" << endl;

   // Next line reminds user 1 hidden layer by definition
   outputStream << "1 hidden layer by definition" << endl;
```

```
   // Next line is number of hidden nodes
   outputStream << nHidden << " hidden nodes" << endl;

   // Remind user 1 output by definition
   outputStream << "1 output node by definition" << endl;
}

// Trains one iteration through the training set, model dependant
//    returns set error
double BareProp::trainSet()
{
   // Easier on the eyes
   unsigned nTrain = theData.getNumTrain(), // examples in training set
      example; // example counter

   double setError = 0; // initialize the set error

   // For off-line training: zero'd accumulator containers for hidden
   //    and output weights--would be expensive, but it's *outside* the
   //    main loop, and so doesn't add much inefficiency to the on-line case
   Matrix< double > hWaccumulate( hW.rows(), hW.cols(), 0 );
   vector< double > oWaccumulate( oW.size(), 0 );

   // Loop through all exemplars in the set
   for ( example = 0; example < nTrain; example++ )
   {
      // Begin by forward propagating the exemplar
      forward( Train, example );

      // Calculate error for single output and add to set error
      errorFunction E( y[ example ], o, x, errorType );
      setError += E.value();

      if ( E.boundsErr() ) // check for out of bounds error in log(o)
         boundsErrorFlag = true;

      // Regularization term for error, Manifest Methodology equation 2.1

      //      1/(2σ_w^2) Σ_{i=1}^m |y|^2 in E_k(y) = e(t^k, a_k^(m)) + 1/(2σ_w^2) Σ_{i=1}^m |y|^2

      if ( weightDecayFlag )
         setError += regularizer * ( hW.squared() + squared( oW ) );

      // Calculate the error term for the output unit
      // (Freeman & Skapura p. 102, #6)

      // δ^o_pk = (y_pk − o_pk) f^o'_k(net^o_pk) for mean squared error

      // Note sign is changed ( o - y ) instead of ( y - o ) to conform to Methodology
      // Methodology equation 2.8 (t=y, a=o)

      // δ_k^(m) = −(t_k − a^(m)) for mean squared error,

      // or equation 2.9

      // δ_k^(m) = −(t_k − a^(m))./[a^(m). ∗ (1 − a^(m)]

      //    for x-entropy error,

      // multiplied by the Df_k^(j) term in equation 2.7

      // Note that if the error is x-entropy, then dE/do = (y-o)/(o(1-o))
      //    and the o(1-o) in the denominator cancels d_sigmoidal

      //    (= f^o'_k, = Df_k^(j)) = o(1-o)
```

```
//    hence splitting this code into 2 lines with an if:
o_err = o - y[ example ];
if ( errorType == 0 ) // error is LMS
   o_err *= d_sigmoidal()( o );

// Calculate the error terms for the hidden units
// (Freeman & Skapura p. 102, #7)
```

// $\delta^h_{pj} = f^{h\prime}_j(net^h_{pj}) \sum_k \delta^o_{pk} w^o_{kj}$

```
// Methodology equation 2.7
```

// $\delta^{(j-1)}_k = [\mathbf{W}^{(j)}]^T \mathbf{Df}^{(j)}_k \delta^{(j)}_k$

```
// Note that using func which takes the output vector as the
//    last argument, and *= instead of *, is the most efficient way
( func( hO, d_sigmoidal(), h_err ) *= o_err ) *= oW;

// Weight decay for canonical backpropagation
```

// $\vec{w}_{t+1} = (1 - 2\eta\lambda)\vec{w}_t - \eta \left.\frac{\partial E}{\partial w}\right|_{w_t}$

```
// and where the gradient doesn't need to be separated
if ( weightDecayFlag && ( trainingType == 0 ) && !gradMaxFlag )
{
   oW *= decayTerm;
   hW *= decayTerm;
}

if ( !batchEpochFlag ) // classic on-line backpropagation
{
   // Tests of gradient calculation were found to slow training
   //    by more than 50%, hence the if block
   if ( ( trainingType == 0 ) && !gradMaxFlag ) // where gradient doesn't need to be separated
   {
      // Update the output weights (Freeman & Skapura p. 102, #8)
```

// $w^o_{kj}(t+1) = w^o_{kj}(t) + \eta \delta^o_{pk} i_{pj}$

```
      // Note that because the output error term is o-y as in Methodology,
      // it will be a subtraction, as in Methodology equation 2.11
```

// $\mathbf{y}_{t+1} = \mathbf{y}_t - \eta \mathbf{g}_k(\mathbf{y}_t)$ where $\mathbf{g}_k = \delta^{(j)}_k \mathbf{Df}^{(j)}_k [\mathbf{f}^{(j-1)}_k]^T$ (Note that Freeman & Skapura's δ^o_{pk} already contains $\mathbf{Df}^{(j)}_k$

```
      oW -= ( hO *= ( eta * o_err ) );
      // Update the hidden weights (Freeman & Skapura p. 102, #9)
```

// $w^h_{ji}(t+1) = w^h_{ji}(t) + \eta \delta^h_{pj} x_i$

```
      // Also Methodology equation 2.11 as above (also a subtraction)
      hW -= ( hWup.outprod( h_err, I ) *= eta );
   }

   else // calculate the gradient as a separate structure
   {
      // Calculate the output and hidden gradients, Methodology equation 2.10
```

// $\mathbf{g}_k = \delta^{(j)}_k \mathbf{Df}^{(j)}_k [\mathbf{f}^{(j-1)}_k]^T$ (Note that Freeman & Skapura's δ already contains $\mathbf{Df}^{(j)}_k$)

```
      oG = ( hO *= o_err ); // *= for efficiency
      hG.outprod( h_err, I );

      // Weight decay, Manifest Methodology section 2.2.1
```

```
            // right hand term (1/σ_w^2)[W , b] in G_k^(j) = δ_k^(j) Df_k^(j) [f_k^(j-1)]^T + (1/σ_w^2)[W , b]
            if ( weightDecayFlag )
            {
               oG += ( oW * decay );
               hG += ( hW * decay );
            }

            // Whatever additional algorithm is chosen
            engine( trainingType, ( iteration * nTrain ) + example );

            // Update the ouput and hidden weights

            // Methodology equation 2.11: y_{t+1} = y_t - η g_k(y_t)
            oW -= ( oG * eta );
            hW -= ( hG * eta );
         }
      }
      else // off-line or batch/epoch learning
      {
         // Where gradient doesn't need to be separated
         if ( ( trainingType == 0 ) && !gradMaxFlag )
         {
            // Accumulate output weight update, see above note
            oWaccumulate += ( hO *= o_err );
            // Accumulate hidden weight update, see above note
            hWaccumulate += hWup.outprod( h_err, I );
         }

         else // calculate the gradient as a separate structure
         {
            // Calculate the ouput and hidden gradients, see above note
            oG = ( hO *= o_err );
            hG.outprod( h_err, I );

            // See above note in on-line block
            if ( weightDecayFlag )
            {
               oG += ( oW * decay );
               hG += ( hW * decay );
            }

            // Update the accumulators

            // Methodology equation 2.14: g = (1/N) Σ_{k=1}^N g_k
            oWaccumulate += oG;
            hWaccumulate += hG;
         }
      }
   } // end of loop for exemplars in training set

   if ( batchEpochFlag ) // off-line or batch/epoch learning
   {
      // Canonical backprop without separate gradient calculation
      if ( ( trainingType == 0 ) && !gradMaxFlag )
      {
         // Batch/epoch updates weights at the end, *now* multiply by eta

         // Methodology equation 2.13: y_{t+1} = y_t - η g(y_t) and 1/N in Methodology equation
2.14: g = (1/N) Σ_{k=1}^N g_k
         oW -= ( ( oWaccumulate *= eta ) /= ( double ) nTrain ); // *=, /= for efficiency
         hW -= ( ( hWaccumulate *= eta ) /= ( double ) nTrain );
      }

      else // routines where gradient is calculated separately
      {
         // Set the gradients to the accumulators so that pack() & unpack() work
```

```
            // 1/N in Methodology equation 2.14: g = (1/N) sum_{k=1}^{N} g_k

            oG = oWaccumulate /= ( double ) nTrain;
            hG = hWaccumulate /= ( double ) nTrain;

            // Whatever additional algorithm is chosen
            engine( trainingType, iteration );

            // Update the ouput and hidden weights

            // Methodology equation 2.13: y_{t+1} = y_t - eta g(y_t)

            oW -= ( oG * eta );
            hW -= ( hG * eta );
         }
      }

      return setError / nTrain; // return the calculated set error
   }

   // Forward propagates for one input vector in dataset, takes dataset Matrix
   //    as first argument, position in dataset as 2nd argument
   void BareProp::forward( Matrix< double >& data, unsigned example )
   {
      // Get a single row from the input Matrix
      // (Freeman & Skapura p. 102, #1)
      data.row( example, I );

      // Take the dot product of the hidden weights Matrix hW and the
      //    transpose of a row of the input Matrix I, and apply the
      //    sigmoidal function to the resulting vector
      // (Freeman & Skapura p. 102, #2 & #3)

      // 2: net^h_pj = sum_{i=1}^{N} w^h_ji x_pi  3: i_pj = f^h_j(net^h_pj)

      func( hW.dotprod( I, hO ), sigmoidal(), hO );

      // Take the dotproduct of the hidden output vector and the output
      //    weights vector, and apply the sigmoidal function to the result
      // (Freeman & Skapura p. 102, #4 & #5)

      // 4: net^o_pk = sum_{j=1}^{L} w^o_kj i_pj  5: o_pk = f^o_k(net^o_pk)

      x = dotprod( hO, oW ); // note that x is inherited from Network
      o = sigmoidal()( x );
   }

   // Remove input nodes from this network, takes vector representing
   //    which nodes to be removed as argument
   void BareProp::removeInputs( const vector< unsigned >& v )
   {
      // Check incoming vector elements in bounds
      assert ( *max_element( v.begin(), v.end() ) < theData.getInput() );

      // Use DataSet::removeInputs to remove inputs in the dataset
      theData.removeInputs( v );

      // Use Model utility function to extract input Matrices
      Model::extractInputMatrices();

      I.resize( theData.getInput() ); // resize the vector for single exemplar inputs

      hW = hW.excludecols( v ); // remove hidden weights corresponding to inputs
      hG = hG.excludecols( v ); // remove hidden gradient columns corresponding to inputs

      hWup.resize( nHidden, theData.getInput() ); // resize hidden weight update Matrix
   }
```

```
// Convert weight gradient structure to single vector stackG
void BareProp::pack()
{
   // Start by converting hidden gradients Matrix to stackG
   stackG = hG.toVector();

   // Then append output gradients vector to stackG
   std::copy( oG.begin(), oG.end(), back_inserter( stackG ) );
}

// Convert single vector stackG back to weight gradient structure
void BareProp::unpack()
{
   vector< double > vpack;  // temporary holding vector
   vpack.resize( theData.getInput() * nHidden );

   // Copy the part of stackG corresponding to the hidden gradients to
   //    its holding vector, which has been sized in setHidden(...)
   vpack.assign( stackG.begin(), stackG.begin() + ( theData.getInput() * nHidden ) );
   toMatrix( hG, vpack ); // convert the holding vector to hidden gradient Matrix

   // Copy the part of stackG corresponding to the output weights to
   //    the output gradients vector
   oG.assign( stackG.begin() + ( theData.getInput() * nHidden ), stackG.end() );
}
```